Aran Learn 4: Probability for Detection

For: everyone reads sections 1 to 3; sections 4 to 6 matter most for whoever builds the detector and the leakage ledger. **Time:** about 6 hours with practice. **Maths needed:** school algebra and logarithms. **Before this:** Learn 2.

Aran never says "this is an Aadhaar" or "this is safe" on a hunch. It gives a number: a probability, a false-alarm rate, a count of bits. This doc teaches the five pieces of probability behind those numbers, each with a worked example from Aran's own data.

1. Probability in plain words

A probability is a number from 0 (never) to 1 (always). It can be written as a fraction, a decimal or a percentage: 1/10 = 0.1 = 10%.

Idea	Notation	Example
Probability of A	P(A)	P(a random 12-digit string passes Verhoeff) = 1/10
A and B both happen, when independent	P(A) × P(B)	Two random strings both pass: 1/10 × 1/10 = 1/100
A does not happen	1 − P(A)	A random string fails: 9/10
A given that B happened	P(A | B)	P(it is a real Aadhaar | it passed Verhoeff)

Conditional probability is the key idea. P(passes Verhoeff | real Aadhaar) = 1, because every real Aadhaar passes. But P(real Aadhaar | passes Verhoeff) can be very small, because most strings that pass are just noise. Mixing these two up is the most common mistake in detection, and it has a name: the **base-rate fallacy**.

Odds are another way to write a probability: the chance it happens against the chance it does not. A probability of 0.2 is odds of 0.2 : 0.8, or 1 : 4. Odds make Bayes' rule very simple, as section 3 shows.

2. The two kinds of mistakes

Every detector makes two kinds of error, and you cannot reduce one without risking more of the other. Aran's job is to trade them off with numbers instead of guesses.

	Really sensitive	**Not sensitive**
Flagged	True positive (TP): caught a real leak	False positive (FP): a false alarm
Not flagged	False negative (FN): a missed leak	True negative (TN): correctly ignored

Metric	Formula	Question it answers
Precision	TP / (TP + FP)	Of everything I flagged, how much was real?
Recall	TP / (TP + FN)	Of everything real, how much did I catch?
False-positive rate	FP / (FP + TN)	Of all the noise, how much did I wrongly flag?
F1 score	2 × P × R / (P + R)	One number balancing precision and recall

Worked example. A batch of logs holds 1,000,000 twelve-digit candidates, of which 1,000 are real Aadhaar numbers. A detector flags 990 of the real ones and 4,000 of the noise.

- TP = 990, FN = 10, FP = 4,000, TN = 995,000.
- Recall = 990 / 1,000 = 99%. Looks excellent.
- False-positive rate = 4,000 / 999,000 = 0.4%. Also looks excellent.
- Precision = 990 / 4,990 = 19.8%. Four out of five alerts are false.
- F1 = 2 × 0.198 × 0.99 / (0.198 + 0.99) = 0.33.

A tiny false-positive rate on a huge pile of noise still buries the real alerts. Analysts who see 80% false alarms start ignoring all alerts. This is exactly what happened with plain pattern matching on the real logs: the OTP regex produced 67,481 hits across 32,000 lines that contain no OTP at all, because every 6-digit timestamp, port and block number looks like an OTP.

3. Bayes' rule: combining evidence

3.1 The odds form

Bayes' rule tells you how to update a belief when new evidence arrives. In odds form it is one multiplication per piece of evidence:

```
posterior odds = prior odds × Λ1 × Λ2 × ...
Λ = P(evidence | real) / P(evidence | noise)
```

Λ (lambda) is the **likelihood ratio**: how many times more likely the evidence is if the value is real than if it is noise. Λ above 1 pushes belief up, below 1 pushes it down, exactly 1 changes nothing. When pieces of evidence are independent, their ratios simply multiply.

3.2 Worked example: is this 12-digit string an Aadhaar?

Suppose 1 in 1,000 twelve-digit strings in a bank log is a real Aadhaar (an assumed number for teaching).

Step	Evidence	Λ	Odds after	Probability after
Start	Prior		1 : 999	0.1%
1	Passes Verhoeff: real always passes, noise passes 1 in 10	1 / 0.1 = 10	10 : 999	1.0%
2	Found in the bank's own-customer registry, false-positive rate 1 in a million	about 1,000,000	about 10,000,000 : 999	99.99%

Two lessons. First, the checksum alone only moves belief from 0.1% to 1%: helpful, not proof. Second, a strong, independent piece of evidence (the registry) turns a hint into near-certainty. That is why Aran's detector is an evidence-fusion detector, not a single rule.

3.3 Evidence can also push down

Context words matter. Suppose that in training data the word "blk_" appears just before 30% of noise numbers but before only 0.03% of real Aadhaar numbers. Then $\Lambda = 0.0003 / 0.3 = 0.001$, and seeing "blk_" divides the odds by 1,000. In practice Aran learns these ratios from data (Learn 5 builds the classifier), and calibration makes sure they are honest probabilities.

3.4 Choosing the threshold: the Neyman-Pearson lemma

After combining evidence you must decide: flag or not? The Neyman-Pearson lemma (1933) says that for any false-alarm rate you are willing to accept, the most powerful test is to flag when the likelihood ratio is above some threshold. No other rule catches more real leaks at the same false-alarm rate. So Aran does not invent ad-hoc rules; it computes the combined ratio and picks the threshold from the false-alarm budget the bank sets, for example "no more than 10 false alerts per day".

4. Filters: knowing what you have seen, without storing it

4.1 Why Aran needs one

The strongest evidence in section 3 was "this number belongs to one of our own customers". But the detector must not keep a plain list of every customer's Aadhaar, because that list would itself be a perfect target. A filter answers "have I seen this?" without storing the items, using HMAC fingerprints (Learn 3) so even the filter's bits reveal nothing without the HSM key.

4.2 Bloom filter, step by step

A Bloom filter is a row of m bits, all 0 at first, and k different hash functions.

- **Add an item:** hash it k ways, giving k positions, and set those bits to 1.
- **Check an item:** look at its k positions. If any bit is 0, the item was definitely never added. If all are 1, it was probably added.

Small worked example with m = 10 bits and k = 2:

Action	Hash positions	Bits after (positions 0 to 9)
Start		0000000000
Add customer A	2, 7	0010000100
Add customer B	4, 7	0010100100
Check C	1, 4	bit 1 is 0, so C is definitely new
Check D	2, 4	both bits are 1, so "probably seen", but D was never added: a false positive

The false positive happened because A and B together set the bits D needs. With n items, the false-positive rate is:

```
error ≈ (1 − e^(−k·n/m))^k
```

For n = 1,000, m = 10,000 and k = 7: error ≈ (1 − e^(−0.7))^7 ≈ 0.0082, a bit under 1%. The best k is (m/n) × ln 2, here about 7, which is why 7 was chosen.

Sizing for a bank. For 100 million identifiers and an error rate of one in a million, the formula m = −n × ln(error) / (ln 2)² gives about 2.9 billion bits, roughly 360 MB, with k = 20. That fits in the memory of one server.

4.3 Why Aran uses a cuckoo filter instead

A Bloom filter cannot delete: clearing a bit might also remove other items that share it. The DPDP right to erasure needs deletion. A cuckoo filter stores a small fingerprint of each item in one of two possible slots, so an item can be removed by deleting its fingerprint. It has similar space use and a similarly predictable error rate.

5. Collisions and the birthday bound

5.1 The birthday problem

In a room of 23 people, the chance that two share a birthday is 50.7%, even though there are 365 days. It feels wrong because we think "how likely is someone to share my birthday?", but the question is about any pair, and 23 people form 253 pairs.

General rule. Drawing N random values from D possibilities, the chance that some two are equal is about N² / (2D) while that number is small. Collisions start to appear around N ≈ √D.

5.2 Why Aran never truncates a hash into a token

A tempting shortcut is "token = the first 11 digits of a hash of the Aadhaar". With 50 million customers (N) and 10^11 possible 11-digit tokens (D), the expected number of colliding pairs is about N² / (2D) = (5 × 10^7)² / (2 × 10^11) = 12,500. That is 12,500 pairs of customers who

would share a token, so their records would be merged. FF1 and the vault are both permutations or lookups with no collisions at all.

5.3 Leak attribution uses the same maths

With per-destination tokens (Mechanism 3), a leaked token is traced to the destination whose token table contains it. Could a token from destination A accidentally also be a valid token of another destination? For S destinations, N customers and D possible tokens, the chance is at most (S − 1) × N / D.

Leaked dump size	Chance of a wrong match (S = 40, N = 50 million, D = 10^11)
1 token	39 × 5×10^7 / 10^11 ≈ 2%
5 tokens, all must match the wrong destination	0.0195^5 ≈ 2.8 × 10^-9, under 3 in a billion

Each extra token multiplies the chance down. The experiment E3 in `research/experiment.py` confirmed 100% correct attribution for dumps of 1, 2, 5 and 50 tokens with 40 destinations.

6. Measuring leakage in bits

6.1 Why bits

To judge a mask, ask: after seeing the masked value, how many possible real values are left? If 1,000 are left, an attacker who can test guesses (for example against a login form or a payment API) needs at most 1,000 tries. Counting in bits makes this easy to compare and add: H = log2(number of values left). Every extra bit doubles the attacker's work.

Values left	Bits	Attacker's work
1,000	10.0	Under a second for a script
100,000	16.6	Seconds
8,000,000	22.9	Minutes
100,000,000,000	36.5	Large, needs many queries

This measure is called **min-entropy**. For values that are equally likely, which is a safe assumption for identifiers, it is exactly log2 of the count:

```
H(X | view) = log2 |{ x : x is consistent with the view }|
```

6.2 Worked example: card shown as first 8 and last 4

A 16-digit card shown as `4111 1111 XXXX 1111` hides 4 digits, so there are 10^4 = 10,000 fillings. But the real number must pass Luhn, and for each choice of the first three hidden digits exactly one choice of the fourth passes. So only 1 in 10 fillings survive: 1,000 values, log2(1,000) ≈ 10 bits.

Aran's code counts these exactly, using dynamic programming over the checksum state instead of trying every filling, so it also works for 12 hidden digits:

```
        from aran_core import count_luhn_completions, count_verhoeff_completions
        known = {0:4, 1:1, 2:1, 3:1, 4:1, 5:1, 6:1, 7:1, 12:1, 13:1, 14:1, 15:1}
        count_luhn_completions(known)                    # 1000
        count_verhoeff_completions({8:2, 9:3, 10:4, 11:6})   # 8000000 (Aadhaar, last 4
shown)
```

The keys of `known` are positions counted from the left starting at 0, and the values are the visible digits.

6.3 The full table, from the research results

Mask	Values left	Bits left
Card 16-digit, last 4 shown	100,000,000,000	36.5
Aadhaar, last 4 shown	8,000,000	22.9
Card 16-digit, first 6 and last 4	100,000	16.6
Card 16-digit, first 8 and last 4	1,000	10.0
Aadhaar, first 4 and last 4	1,000	10.0

Card networks have moved to 8-digit IINs, and the PCI Security Standards Council allows showing the first 8 and last 4 digits of a 16-digit card. That display leaves only 10 bits, compared with 16.6 bits for the older first 6 and last 4.

6.4 The leakage ledger

Masks are usually judged one system at a time, but data gets combined. Suppose the CRM shows the first 8 digits of a card and the call-centre tool shows the last 4. Alone, the CRM view leaves 10^7 possibilities and the call-centre view leaves 10^11. If one person can see both screens, together they leave 1,000.

Aran's leakage ledger (Mechanism 4) records what each destination sees for each field. For every group of destinations that can share data, it computes the bits left when their views are combined, and it blocks a new data flow that would drop any customer's secret below a threshold, such as 20 bits. The check runs before the flow is allowed, not after a leak.

7. Practice

- **P1.** Compute log2(8,000,000) with Python's `math.log2` and compare with 22.9 bits.
- **P2.** In the precision example of section 2, how many false positives can the detector allow if you want 90% precision with the same 990 true positives?
- **P3.** Redo the Bayes example in 3.2 with a prior of 1 in 100. What is the final probability? (Answer: odds 1 : 99, then 10 : 99, then about 10^7 : 99, so 99.999%.)
- **P4.** Use the Bloom formula for n = 1,000, m = 10,000, k = 3 and k = 12. Which is worse, and why does too many hash functions hurt?

- [ ] **P5.** Simulate the birthday problem: draw 23 random days 10,000 times and count how often two match. You should get close to 50.7%.
- [ ] **P6.** Use `count_luhn_completions` to find the bits left for a 16-digit card showing the first 6 and last 2. Is that mask safer than first 6 and last 4?
- [ ] **P7.** Write a tiny ledger: a dict from destination to the set of visible card positions. For a pair of destinations, union their positions and compute the bits left.

Hint for P2. Precision = 990 / (990 + FP) = 0.9 gives FP = 110.

8. Further reading

- Khan Academy's free probability unit: conditional probability and Bayes' theorem.
- 3Blue1Brown, "Bayes theorem, the geometry of changing beliefs" (YouTube).
- Mitzenmacher and Upfal, "Probability and Computing", chapter 5, for Bloom filters and the birthday problem.
- Fan et al., "Cuckoo Filter: Practically Better Than Bloom" (CoNEXT 2014).
- Geoffrey Smith, "On the Foundations of Quantitative Information Flow" (2009), for min-entropy.